

CCES – College of Computer Engineering and Sciences

Prince Sattam Bin Abdulaziz University

CS3401 – Algorithm Design and Analysis

Chapter 4

Decrease & Conquer and Divide & Conquer

Academic Year 2026 – 2027



Dr. Khalil Chebil



Dr. Esraa

Chapter Overview

Recursive Reduction — A Unified View

Decrease by a Constant

Decrease by a Constant Factor

Variable-Size Decrease

Divide & Conquer — Introduction

Mergesort

Quicksort

Binary Trees & Closest Pair

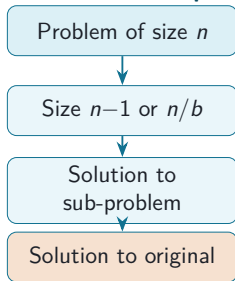
Section 1

Recursive Reduction — A Unified View

Two Sides of the Same Coin

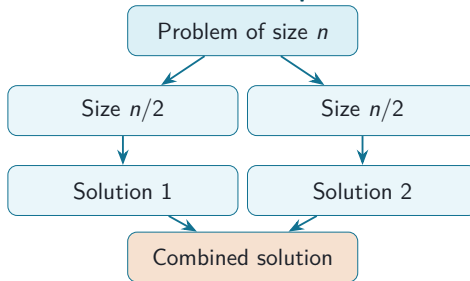
Both strategies rely on **recursive reduction**: *solve a large instance by solving one or more smaller instances.*

Decrease & Conquer



ONE smaller sub-problem

Divide & Conquer



TWO (or more) equal sub-problems + combine step

Three Variants of Decrease & Conquer

1. Decrease by a constant (usually by 1)

$$T(n) \rightarrow T(n-1) + \text{work}$$

- ▶ Insertion sort, topological sort
- ▶ Generating permutations and subsets

2. Decrease by a constant factor (usually by 2)

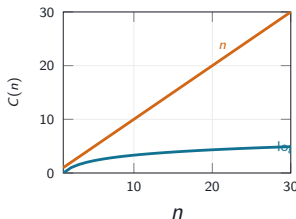
$$T(n) \rightarrow T(n/2) + \text{work}$$

- ▶ Binary search, fake-coin problem
- ▶ Exponentiation by squaring, Russian peasant mult.

3. Variable-size decrease

$$T(n) \rightarrow T(n') + \text{work}, \text{ where } n' \text{ varies}$$

- ▶ Euclid's GCD, Quickselect, BST operations



Decrease-by-factor $\Rightarrow \mathcal{O}(\log n)$

Decrease-by-1 $\Rightarrow \mathcal{O}(n)$ (or $\mathcal{O}(n \log n)$)

Section 2

Decrease by a Constant

Insertion Sort

Idea: To sort $A[0..n-1]$, first sort $A[0..n-2]$, then *insert* $A[n-1]$ into the sorted part.

Algorithm

```
1: for  $i \leftarrow 1$  to  $n-1$ 
2:    $v \leftarrow A[i]; j \leftarrow i - 1$ 
3:   while  $j \geq 0$  and  $A[j] > v$ 
4:      $A[j+1] \leftarrow A[j]; j \leftarrow j - 1$ 
5:   end while
6:    $A[j+1] \leftarrow v$ 
7: end for
```

Trace on $[5, 3, 8, 1]$:

5 3 8 1
3 5 8 1
3 5 8 1
1 3 5 8

Comparisons

Best (sorted)	$n - 1$	$\Theta(n)$
Worst (reverse)	$n(n - 1)/2$	$\Theta(n^2)$
Average	$n^2/4$	$\Theta(n^2)$
Stable: Yes In-place: Yes		

Topological Sorting

Definition

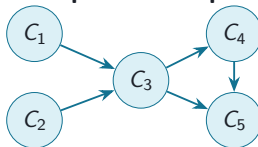
A **topological sort** of a DAG is a linear ordering of its vertices such that for every directed edge (u, v) , u appears before v .

Source-removal algorithm:

1. Compute in-degree for all vertices
2. Enqueue all vertices with in-degree = 0
3. Repeat until queue is empty:
 - Dequeue vertex v , add to sorted list
 - Decrease in-degree of all neighbors of v
 - Enqueue neighbors whose in-degree becomes 0
4. If sorted list has $< n$ vertices: **cycle detected**

Complexity: $\mathcal{O}(|V| + |E|)$

Example: course prerequisites



Result: C_1, C_2, C_3, C_4, C_5

In-degrees:

$C_1=0, C_2=0, C_3=2, C_4=1, C_5=2$

Generating Subsets

Key observation: Every subset of $\{a_1, \dots, a_n\}$ either contains a_n or it does not.

Recurrence:

$$\text{Subsets}(n) = \text{Subsets}(n-1) \cup \{S \cup \{a_n\} \mid S \in \text{Subsets}(n-1)\}$$

Bit-string representation: Each subset $\leftrightarrow$ a binary string of length n (1 = included, 0 = excluded).

Generate binary numbers from 0 to $2^n - 1$.

All subsets of $\{a_1, a_2, a_3\}$:

Bits	Subset
000	$\emptyset$
001	$\{a_3\}$
010	$\{a_2\}$
011	$\{a_2, a_3\}$
100	$\{a_1\}$
101	$\{a_1, a_3\}$
110	$\{a_1, a_2\}$
111	$\{a_1, a_2, a_3\}$

Total: $2^3 = 8$ subsets

$\mathcal{O}(2^n)$

Section 3

Decrease by a Constant Factor

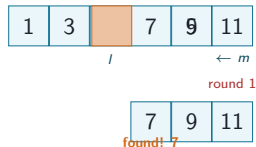
Binary Search

Problem: Find key K in a *sorted* array $A[0..n-1]$.

Algorithm

```
1:  $l \leftarrow 0$ ;  $r \leftarrow n - 1$ 
2: while  $l \leq r$ 
3:    $m \leftarrow \lfloor (l + r) / 2 \rfloor$ 
4:   if  $K = A[m]$  then return  $m$ 
5:   else if  $K < A[m]$  then
6:      $r \leftarrow m - 1$ 
7:   else
8:      $l \leftarrow m + 1$ 
9:   end if
10: end while
11: return  $-1$ 
```

Trace: search for 7 in $[1, 3, 5, 7, 9, 11]$



$$T(n) = T(n/2) + \mathcal{O}(1) \implies \mathcal{O}(\log n)$$

Fake Coin Problem

Problem: Among n identical-looking coins, one is counterfeit (lighter). Find it using a balance scale.

Decrease-by-2 (halving):

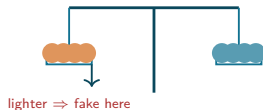
- ▶ Split coins into two equal groups
- ▶ Weigh them — the lighter group contains the fake
- ▶ Recurse on the lighter half

$$W(n) = W(\lfloor n/2 \rfloor) + 1 \implies W(n) = \lfloor \log_2 n \rfloor$$

Decrease-by-3 (thirds) — even better:

- ▶ Split into three equal groups; weigh two of them
- ▶ One weighing eliminates 2/3 of candidates

$$W(n) = W(\lceil n/3 \rceil) + 1 \implies W(n) = \lceil \log_3 n \rceil$$



Key insight

Decrease-by-3 requires $\lceil \log_3 n \rceil$ weighings, fewer than $\lfloor \log_2 n \rfloor$ for $n > 3$.

Russian Peasant Multiplication

Problem: Compute $m \times n$ using only halving, doubling, and addition.

Recurrence:

$$m \cdot n = \begin{cases} \frac{m}{2} \cdot 2n & m \text{ even} \\ \frac{m-1}{2} \cdot 2n + n & m \text{ odd} \end{cases}$$

Add n whenever m is *odd*; always halve m and double n .

Example: 20×26

m	n	add?
20	26	—
10	52	—
5	104	+104
2	208	—
1	416	+416

Result = 104 + 416 = 520

$\mathcal{O}(\log n)$

Section 4

Variable-Size Decrease

Quickselect — Finding the k -th Smallest

Problem: Find the k -th smallest element in an unsorted array of n elements (e.g., the *median*).

Lomuto partition around pivot p (first element):

$$\underbrace{A[0] \cdots A[s-1]}_{\leq p} \quad p \quad \underbrace{A[s+1] \cdots A[n-1]}_{\geq p}$$

After partition, pivot is at index s :

- ▶ If $s = k - 1$: **done**, return $A[s]$
- ▶ If $s > k - 1$: recurse on left part
- ▶ If $s < k - 1$: recurse on right part

Complexities:

Case	Complexity
Best	$\Theta(n)$
Worst	$\Theta(n^2)$
Average	$\Theta(n)$

Real-world use

Finding the median salary among 1 million records *without* sorting them all — used in statistics, databases, and data science pipelines.

Interpolation Search

Idea: Like binary search, but probe position is estimated from the *value* of the key, assuming uniform distribution.

Algorithm sketch

```
1: while  $l \leq r$  and  $v \in [A[l], A[r]]$ 
2:   compute probe position  $x$  via formula
3:   if  $A[x] = v$  then return  $x$ 
4:   else if  $A[x] < v$  then
5:      $l \leftarrow x + 1$ 
6:   else
7:      $r \leftarrow x - 1$ 
8:   end if
9: end while
10: return  $-1$ 
```

$$x = l + \left\lfloor \frac{(v - A[l])(r - l)}{A[r] - A[l]} \right\rfloor$$

Case	Complexity
Best (uniform)	$\mathcal{O}(\log \log n)$
Worst	$\mathcal{O}(n)$
Average (uniform)	$\mathcal{O}(\log \log n)$

When to prefer over binary search

Only for *very large* sorted arrays with *uniformly distributed* values. Otherwise binary search is safer.

Section 5

Divide & Conquer — Introduction

Divide & Conquer: The General Recurrence

Recurrence

$$T(n) = a T(n/b) + f(n), \quad f(n) \in \Theta(n^d), d \geq 0$$

Master Theorem

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d \text{ (combine dominates)} \\ \Theta(n^d \log n) & \text{if } a = b^d \text{ (balanced)} \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \text{ (recursion dominates)} \end{cases}$$

Recurrence	a	b	d	Case	Result
$3T(n/2) + n$	3	2	1	$a > b^d$	$\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$
$3T(n/2) + n^2$	3	2	2	$a < b^d$	$\Theta(n^2)$
$4T(n/2) + n^2$	4	2	2	$a = b^d$	$\Theta(n^2 \log n)$
$2T(n/2) + n$	2	2	1	$a = b^d$	$\Theta(n \log n)$

Does not apply when:

- ▶ $a < 1$ or $d < 0$
- ▶ $f(n)$ is not polynomial
- ▶ a is not a constant

Memory aid

Compare the *branching factor* a with the *shrink-work ratio* b^d :

- ▶ $a < b^d$: work at root dominates
- ▶ $a = b^d$: cost spreads evenly
- ▶ $a > b^d$: leaves dominate

Section 6

Mergesort

Mergesort — Algorithm

Idea: Split array in half, sort each half recursively, then *merge* the two sorted halves.

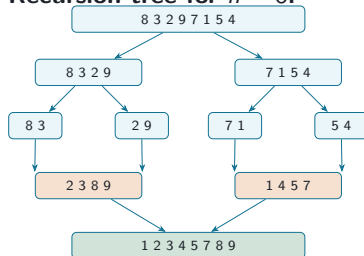
Mergesort

Input: Array $A[0..n-1]$

- 1: **if** $n > 1$ **then**
- 2: split A into $B[0..\lfloor n/2 \rfloor - 1]$ and $C[...]$
- 3: MERGESORT(B)
- 4: MERGESORT(C)
- 5: MERGE(B, C, A)
- 6: **end if**

Merge: compare heads of B and C , copy smaller.
Runs in $\Theta(p + q)$ where $|B| = p$, $|C| = q$.

Recursion tree for $n = 8$:



Mergesort — Analysis

Recurrence:

$$T(n) = 2T(n/2) + \Theta(n), \quad T(1) = \mathcal{O}(1)$$

Master Theorem: $a = 2, b = 2, d = 1 \implies a = b^d$

$T(n) = \Theta(n \log n)$

All cases are $\Theta(n \log n)$ (the merge always takes $\Theta(n)$).

Number of comparisons (worst case) $\approx n \log_2 n$, close to the information-theoretic lower bound

$$\lceil \log_2 n! \rceil \approx n \log_2 n - 1.44n.$$

	Mergesort		Bubble
Best	$\Theta(n \log n)$	$\Theta(n)$	
Worst	$\Theta(n \log n)$	$\Theta(n^2)$	
Average	$\Theta(n \log n)$	$\Theta(n^2)$	
Space	$\Theta(n)$	$\Theta(1)$	
Stable	Yes	Yes	

	Mergesort		Bubble
Best	$\Theta(n \log n)$	$\Theta(n)$	
Worst	$\Theta(n \log n)$	$\Theta(n^2)$	
Average	$\Theta(n \log n)$	$\Theta(n^2)$	
Space	$\Theta(n)$	$\Theta(1)$	
Stable	Yes	Yes	

Trade-off

Mergesort is not *in-place* — it requires $\Theta(n)$ extra space for the merge step.
Prefer it for *linked lists*.



Section 7

Quicksort

Quicksort — Algorithm

Idea: Choose a *pivot*, partition the array so that all elements $\leq$ pivot come first, then recursively sort both parts.

Quicksort

Input: Subarray $A[l..r]$

```
1: if  $l < r$  then
2:    $s \leftarrow \text{PARTITION}(A[l..r])$ 
3:    $\text{QUICKSORT}(A[l..s-1])$ 
4:    $\text{QUICKSORT}(A[s+1..r])$ 
5: end if
```

Hoare partition scans inward from both ends, swapping out-of-place elements until the pointers cross.

Pivot strategies:

- **First element** — simple, **bad on sorted input**
- **Median-of-three** — good average behaviour
- **Random element** — avoids adversarial inputs

Complexities:

Best / Average	$\Theta(n \log n)$
Worst	$\Theta(n^2)$
Space (avg)	$\mathcal{O}(\log n)$ stack
Stable	No
In-place	Yes

Quicksort vs Mergesort

Property	Quicksort	Mergesort
Average time	$\Theta(n \log n)$	$\Theta(n \log n)$
Worst time	$\Theta(n^2)$	$\Theta(n \log n)$
Extra space	$\mathcal{O}(\log n)$	$\Theta(n)$
In-place	Yes	No
Stable	No	Yes
Cache perf.	Better	Good

Guideline

- ▶ Prefer **Quicksort** for arrays in practice (better cache behaviour, in-place)
- ▶ Prefer **Mergesort** for linked lists (in-place merge in $\mathcal{O}(1)$ extra space)
- ▶ Use **Timsort** (hybrid) for general-purpose libraries (Python, Java)

Section 8

Binary Trees & Closest Pair

Binary Tree Traversals

A **binary tree** is naturally processed by Divide & Conquer: divide into root + left subtree + right subtree.

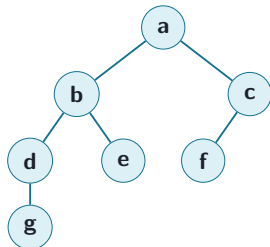
Height of a binary tree

```
1: if  $T = \emptyset$  then return -1
2: end if
3: return  $\max(\text{Height}(T_L), \text{Height}(T_R)) + 1$ 
```

Traversal orders:

- ▶ **Preorder**: root, left, right
- ▶ **Inorder**: left, root, right $\leftarrow$ produces sorted order for BST
- ▶ **Postorder**: left, right, root

All traversals: $\Theta(n)$



Preorder: a, b, d, g, e, c, f

Inorder: g, d, b, e, a, f, c

Postorder: g, d, e, b, f, c, a

Closest Pair — Divide & Conquer

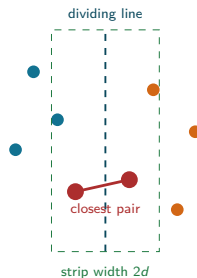
Problem: Find the two closest points in n points in the plane. Brute force: $\mathcal{O}(n^2)$.

D&C algorithm sketch:

1. Sort points by x -coordinate
2. Split at median x into P_L and P_R
3. Recurse: $d_L = \text{closestPair}(P_L)$,
 $d_R = \text{closestPair}(P_R)$
4. $d = \min(d_L, d_R)$
5. Check the "strip" of width $2d$ around the dividing line (at most 8 point-pairs per strip element)

Recurrence: $T(n) = 2T(n/2) + \mathcal{O}(n)$

$\implies T(n) = \mathcal{O}(n \log n)$



Chapter 4 Summary

Decrease & Conquer:

Insertion sort $\mathcal{O}(n^2)$ worst
Topological sort $\mathcal{O}(|V| + |E|)$
Binary search $\mathcal{O}(\log n)$
Quickselect $\mathcal{O}(n)$ avg

Divide & Conquer:

Mergesort $\Theta(n \log n)$ all cases
Quicksort $\Theta(n \log n)$ avg
Closest pair $\Theta(n \log n)$
Tree traversal $\Theta(n)$

Master Theorem

$$T(n) = aT(n/b) + \Theta(n^d):$$

- ▶ $a < b^d$: $\Theta(n^d)$
- ▶ $a = b^d$: $\Theta(n^d \log n)$
- ▶ $a > b^d$: $\Theta(n^{\log_b a})$

Next chapter

Chapter 5: Transform & Conquer — presorting, balanced trees, heaps, and problem reduction (including linear programming).